Infrastructure as Code Foundations

Part 1 — From cloud strategy to IaC architecture

SEAtS | 424812

University of Southampton

Intended Learning Outcomes

By the end of this session you will be able to:

1. Define cloud computing (NIST) and explain how cloud infrastructure has evolved.
2. Compare cloud deployment models, and on-premises vs cloud computing.
3. Distinguish IaaS, PaaS and SaaS by who manages which layer of the stack.
4. Explain when to apply service-communication, data-flow, and processing/control architectural patterns.
5. Explain how DevOps and "cattle not pets" thinking enable reproducible infrastructure.
6. Compare manual, ad hoc and IaC approaches, and define IaC and its requirements.
7. Decompose infrastructure into modules and compare declarative and imperative approaches.

© University of Southampton

Agenda

1. Cloud context, evolution, and deployment models.
2. Service models: who manages what.
3. Architectural patterns.
4. As-a-Service, cattle vs pets, DevOps.
5. The problem IaC solves; definition and requirements.
6. System architecture for IaC.
7. Summary.

© University of Southampton

What is Cloud Computing?

> “A model for enabling convenient, on-demand network access to a shared pool of configurable computing resources… that can be rapidly provisioned and released with minimal management effort or service provider interaction.” — NIST SP 800-145.

1. On-demand self-service
2. Broad network access
3. Resource pooling
4. Rapid elasticity
5. Measured service

- Improved security and “unlimited” capacity.
- Cost efficiencies; zero up-front capital.
- Elastic scaling as system grows.
- Time efficiencies via virtual resources.
- Power efficiency and visibility into consumption.
- Improved process control and quick recovery.

© University of Southampton

timeline title Evolution of Cloud Infrastructure Wave 1 : Static infrastructure : Single VMs accessed via SSH Wave 2 : Growing complexity : Containers : Provisioning tools (Docker, Datadog) Wave 3 : Modern cloud : Containers : Serverless : Managed services

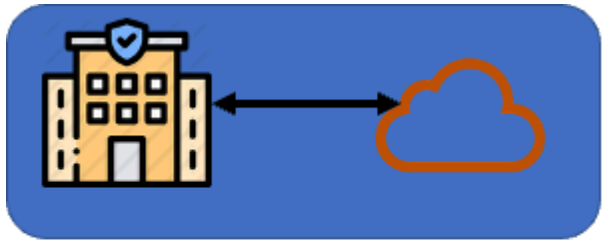
© University of Southampton

Each wave adds more moving pieces — and more edges connecting them. **IaC is the glue between those edges.** Counter-intuitively, a more complex-looking modern architecture diagram can be *easier* to maintain than an old simple one, because the slice of it that's actually your own hand-written code (as opposed to managed service configuration) keeps shrinking — less operational burden sits on your team as the cloud provider absorbs more of the plumbing.

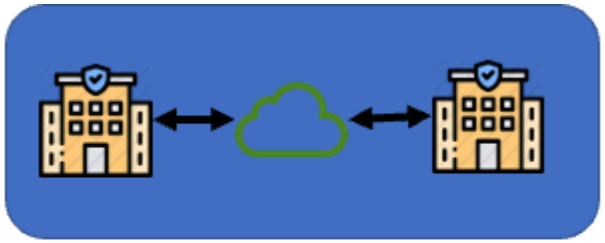
© University of Southampton

Cloud Deployment Models

1. **Public:** Infrastructure available to general public, owned by service provider.
2. **Private:** Dedicated to a single organisation, managed on/off premise.
3. **Community:** Shared by organisations with common concerns.
4. **Hybrid:** Combination of cloud types.



Private



Public



Hybrid



Community

© University of Southampton

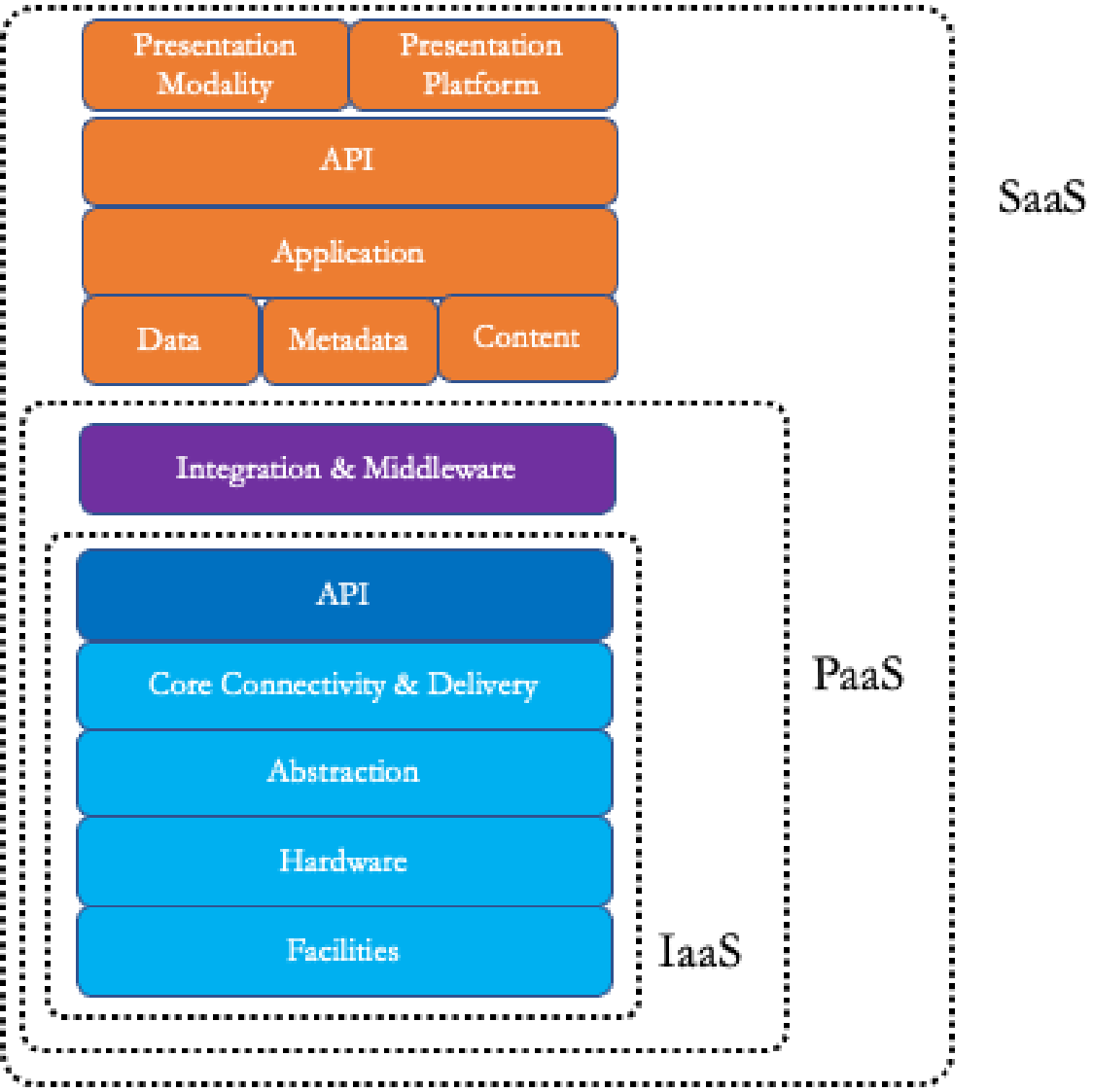
On premises Computing vs. Cloud Computing

On Premises	Cloud
Manually provisioned	Self-provisioned
Dedicated hardware	Shared hardware
Fixed capacity	Elastic capacity
Pay for capacity	Pay for use
CapEx + OpEx	Operational expenses
Managed via sysadmins	Managed via APIs



© University of Southampton

Main Service Delivery Models



IaaS: Provision virtual machines, networks, and storage inside the provider's infrastructure to run any OS or application stack.

- Control over guest OS and runtimes.
- Examples: EC2, Virtual Networks.

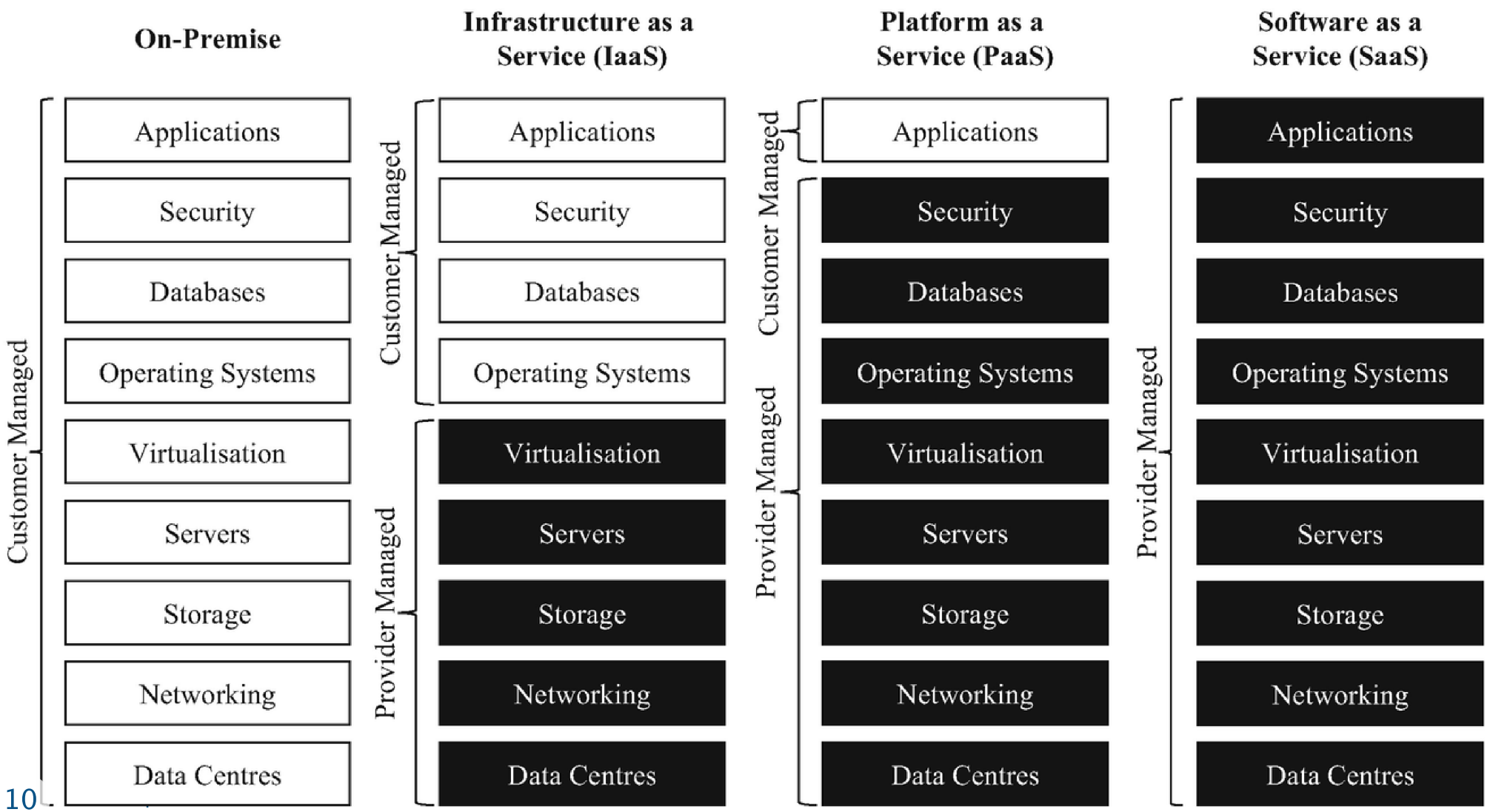
PaaS: Build and deploy custom applications using provider-supported runtimes and tooling.

- Auto elasticity and built-in CI/CD.
- Examples: App Engine, Azure App Service.

SaaS: Consume fully managed provider applications delivered over the cloud.

- No infrastructure management; pay per use.
- Examples: Gmail, Salesforce.

© University of Southampton



© University of Southampton
University of Southampton

Service Delivery Models — Who Manages What

IaaS: You manage from the OS up — runtime, middleware, data, and the application.

PaaS: You manage your application and data; the provider manages the OS, runtime, and middleware.

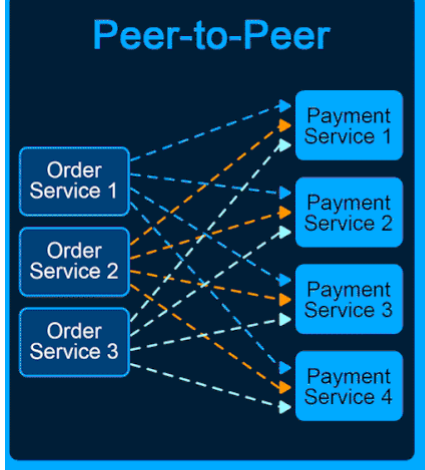
SaaS: The provider manages everything — you just use the application.

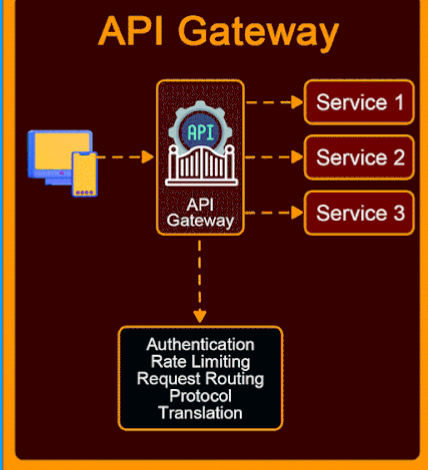
IaC's job: write the code that manages the **IaaS resources** — and the SaaS-like managed services — a cloud provider gives you. The inner the layer, the more of it your Pulumi/Terraform code is responsible for.

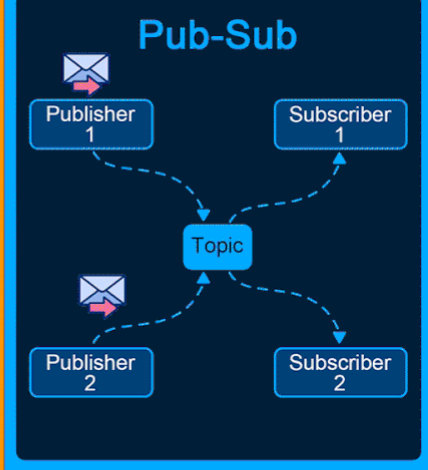
© University of Southampton

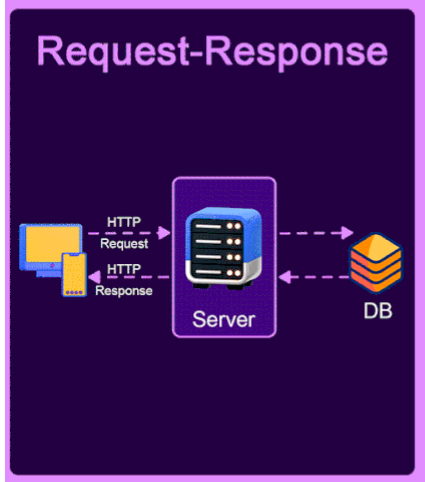
Architectural Patterns

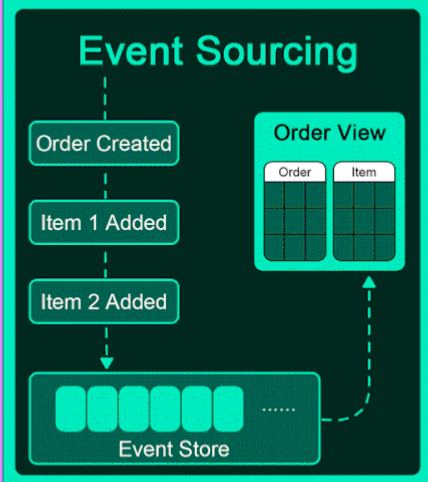
Pattern	Description
Peer-to-Peer	Direct communication between two components without a central coordinator.
API Gateway	Single entry point for all client requests to the backend services of an application.
Pub-Sub	Decouples publishers from subscribers through a message broker.
Request-Response	Client sends a request to a server and waits for a response.
Event Sourcing	Store application state changes as a sequence of immutable events.
ETL	Gather data from sources, transform it, and load into a destination.
Batching	Accumulate data over time/threshold before processing as a group.
Streaming Processing	Continuously ingest, process, and analyse data in real time.
Orchestration	Central coordinator manages interactions between distributed components.

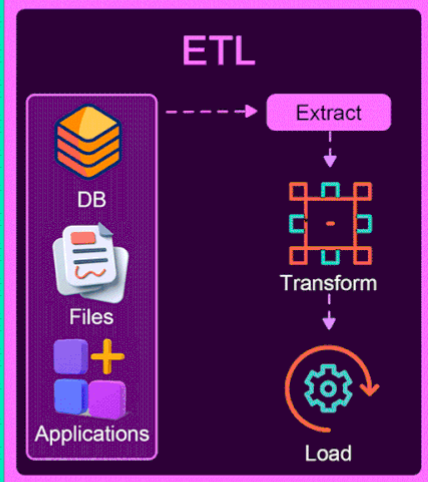


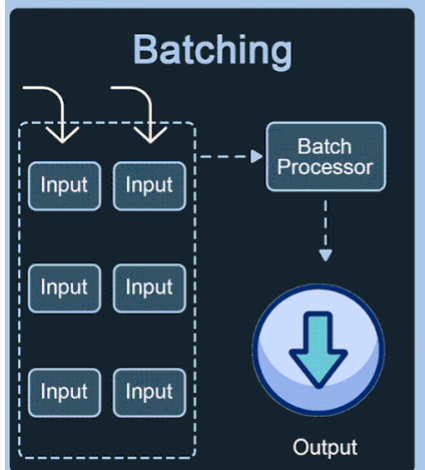


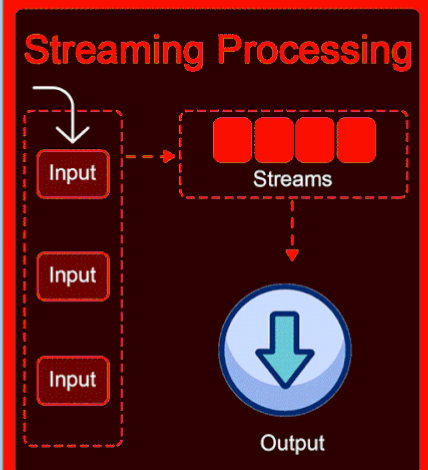


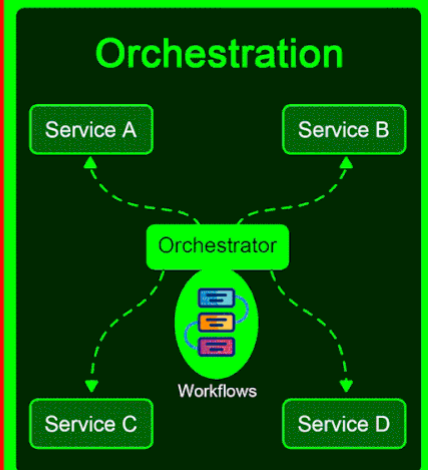












© University of Southampton

Architectural Patterns — Service Communication (1/3)

Peer-to-Peer

```
flowchart LR
O1["Order Svc 1"] -.-> P1["Payment Svc 1"]
O1 -.-> P2["Payment Svc 2"]
O2["Order Svc 2"] -.-> P2
O2 -.-> P3["Payment Svc 3"]
O3["Order Svc 3"] -.-> P3
O3 -.-> P4["Payment Svc 4"]
```


© University of Southampton

API Gateway

```
flowchart LR Client --> "] GW --> "] GW --> "] GW --> "] GW -.-> Cross["Auth ·  "]
GW["API Gateway  S1["Service 1  S2["Service 2  S3["Service 3  Rate Limit · Routing
```


© University of Southampton

Pub-Sub

```
flowchart LR
Pub1["Publisher 1
"] --> Topic(("Topic
")) Pub2["Publisher 2
"] --> Topic Topic --> Sub1["Subscriber 1
"] Topic --> Sub2["Subscriber 2
"]
```


© University of Southampton

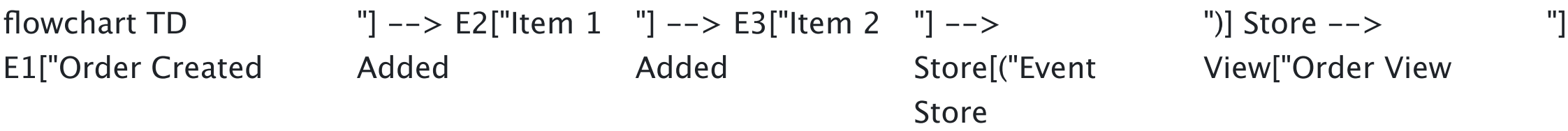
Architectural Patterns — Data Flow (2/3)

Request-Response

flowchart LR Client -- "HTTP Request " --> Server Server -- "HTTP Response " --> Client Server <--> DB[("Database ")]

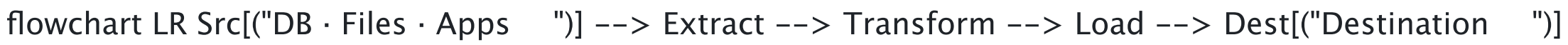
© University of Southampton

Event Sourcing



© University of Southampton

ETL



© University of Southampton

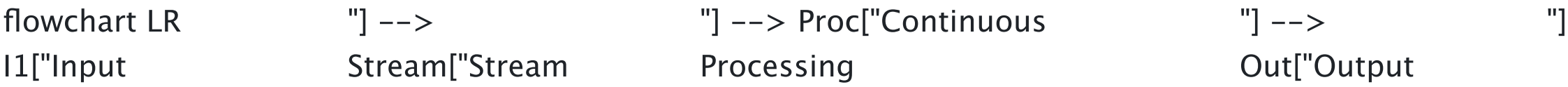
Architectural Patterns — Processing & Control (3/3)

Batching

flowchart LR I1["Input "] & I2["Input "] & I3["Input "] --> BP["Batch Processor "] --> Out["Output "]

© University of Southampton

Streaming Processing



© University of Southampton

Orchestration

```
flowchart TD
Orch(("Orchestrator
")) -->
A["Service A
"] Orch -->
B["Service B
"] Orch -->
C["Service C
"] Orch -->
D["Service D
"]
```


© University of Southampton

Many “as-a-Service” Offerings

Examples include Analytics, API, AI, Backend, Banking, Big Data, Blockchain, Business Process, Code, Content, Construction, Container, Crane, Edge Compute, Contact Centre, Communications Platform, Data, Desktop, Drone, Database, Distribution, Energy, Engagement, EV, Function, Farming, Firewall, Games, Governance, Hadoop, Hardware, Housing, Identity, Image, IoT, IT, Knowledge, Load Balancing, Logging, Management, Microgrid, Mobility, Monitoring, Metal, Mobile Backend, Machine Learning, Network, Network Defense, Payments, Policy, Product, Push Notification, QA, RAN, Recovery, Recycling, Robot, Search, Security, Solar, Shared Storage, Transportation, Testing, Time Series DB, Unified Communications, and “Anything as a Service.”

© University of Southampton

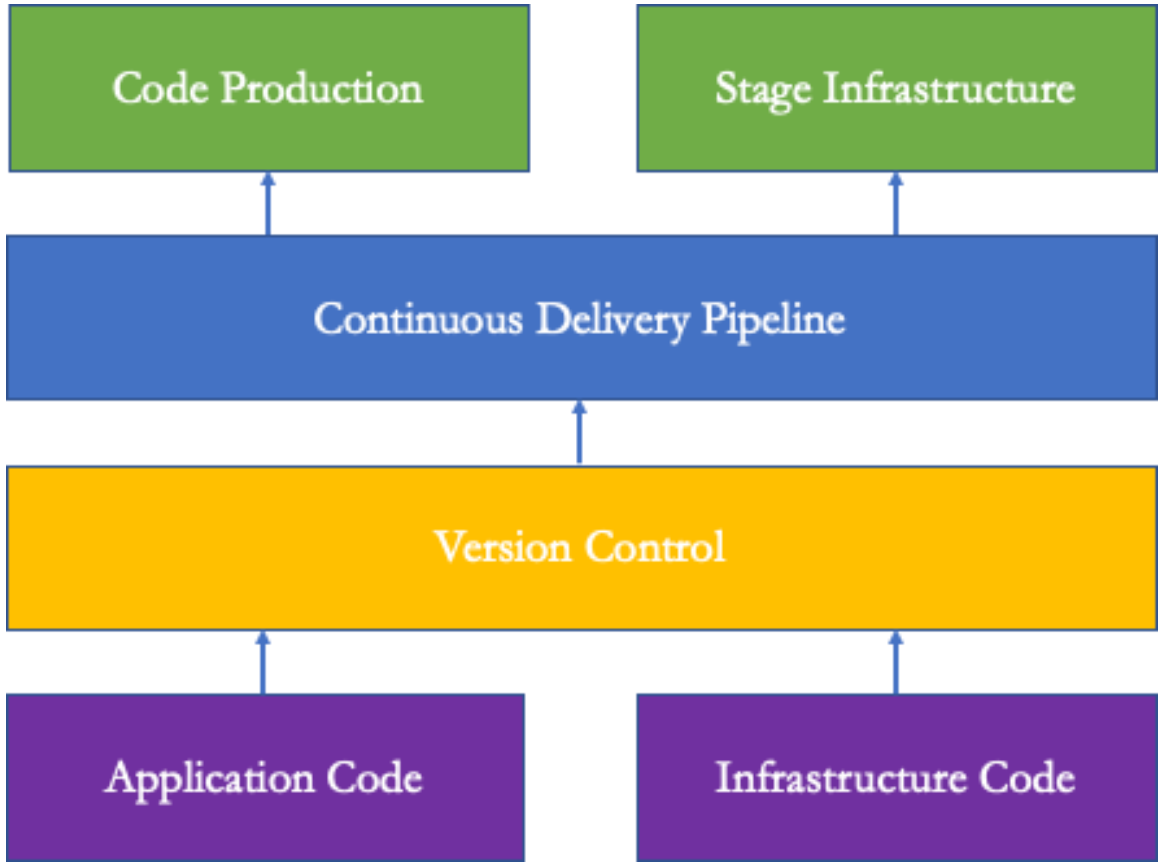
Infrastructure: Cattle vs Pets

Development environments should be consistent and reproducible, like managing “cattle” (identical, disposable instances) rather than unique “pets” (customised desktops). This ensures faster onboarding, predictable testing, and reliable automation.



© University of Southampton

DevOps =



Development + Operations

Development view: Focus on delivering features quickly with automated builds, tests, and infrastructure definitions.

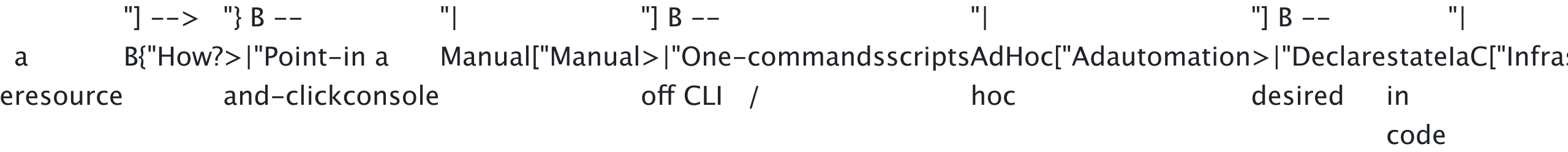
Operations view: Focus on reliability, monitoring, capacity planning, and incident response.

DevOps bridge: Shared tooling (CI/CD, IaC), common metrics (SLOs), and collaborative culture align both perspectives.

© University of Southampton

Naming the Problem IaC Solves

Modern applications can deploy to production up to **100 times a day**, and infrastructure often needs to scale up or down automatically in response to load — neither is viable with manual, one-off changes.



© University of Southampton

Three Ways to Manage Infrastructure

Approach	What it is
Manual	Point-and-click resource creation/modification directly in a console
Ad hoc automation	CLI commands or scripts — some code, but not structured for reuse
Infrastructure as Code	**Provisioning** (declaratively creating/modifying resources) + **configuration** (changing an existing resource's state)

© University of Southampton

IaC Definition & Motivation

Infrastructure as Code (IaC): Define and manage infrastructure via machine-readable files instead of manual provisioning.

Why essential?

- Rapid provisioning for software, hardware, and learning integration.
- Reduces configuration drift and enables CI/CD pipelines.
- Enhances repeatability, testing, and formal specification.
- Critical for distributed cloud systems.

© University of Southampton

Requirements in IaC

Functional examples: Deploy VPC/VNet; automate Kubernetes clusters; create IAM roles.

Non-functional examples: Scalability, security hardening, fault tolerance, cost-efficiency, auditability, compliance.

IaC formalises both sets of requirements in code.

- Cloud provider selection (AWS, Azure, GCP).
- State storage (S3, Blob, GCS).
- Secrets management (Vault, Secrets Manager).
- Access patterns (SSH, API gateways, private endpoints).
- Compliance constraints (residency, logging).
- Integration requirements (CI, monitoring, networks).

Methods: stakeholder interviews, architecture diagrams, threat modelling.

© University of Southampton

System Architecture for IaC

IaC encourages decomposition into modules:

- Networking (VPC, subnets, firewalls).
- Compute (VMs, containers).
- Storage (S3, RDS).
- Identity (IAM roles, policies).
- Monitoring (metrics, logging).

Declarative vs Imperative

Approach	Description	Examples
Declarative	Describe desired state; engine reconciles	Terraform, CloudFormation, Bicep
Imperative	Execute procedural steps	Ansible, Pulumi, scripts

Declarative scales better for large multi-component systems.

© University of Southampton

Summary

- Cloud computing delivers on-demand, elastic, pay-per-use resources; each wave of cloud adds moving pieces, and IaC is the glue between them.
- Deployment models (public, private, community, hybrid) and service models (IaaS, PaaS, SaaS) set what you manage and what the provider manages.
- Architectural patterns group into service communication (API Gateway, Pub-Sub), data flow (Event Sourcing, ETL), and processing & control (Streaming, Orchestration).
- DevOps and "cattle not pets" demand consistent, disposable, reproducible environments.
- IaC replaces manual and ad hoc changes with machine-readable definitions — reducing drift and enabling CI/CD.
- Capture functional and non-functional requirements, decompose into modules, and prefer declarative definitions for large systems.

© University of Southampton